

{EPITECH}

# TIME MANAGER

< TRINITY />



# TIME MANAGER



We don't evaluate what you can copy-paste, but what you **UNDERSTAND** and **WHY** you do it.

## Context: Trinity Market

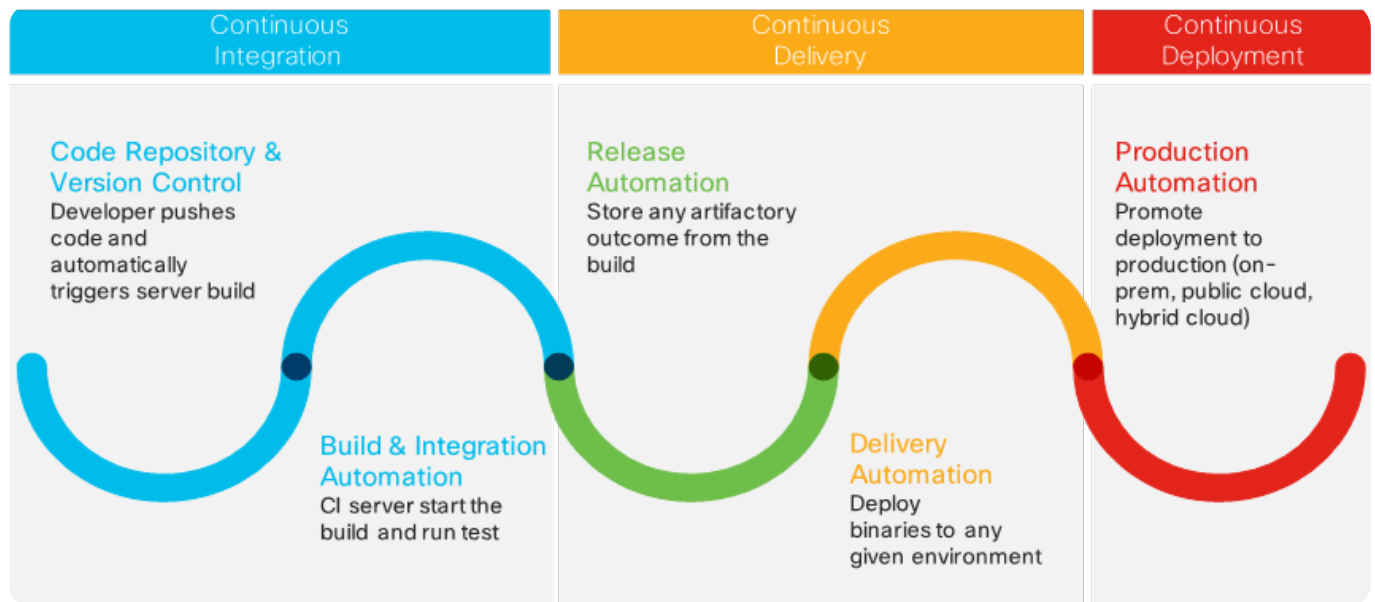
Trinity Market is a digital retail company with:

- ✓ 600 employees: hybrid work, remote + office
- ✓ 5 departments: HR, Finance, IT, Operations, Customer Service
- ✓ Growing fast: 2000+ employees expected in 3 years
- ✓ Strict compliance: GDPR + labor law

The company grew rapidly; time management and supervision became chaotic, with numerous negative impacts on the business. Today, a digital solution is eagerly awaited to ease the company's burden.

## Following the DevOps philosophy

Since you are the sharpest tool in the box, you take the excellent initiative of implementing the [DevOps methodology](#) for this new project.



Doing so, you will be more agile, reduce your delivery delays through automation that accelerates development and ensure the quality of your project.

Before starting to develop your software, consider **which technologies to use and how to structure the project**.

Once you have defined them, the next step is to implement some good DevOps practices. Setting up your environments is crucial, as is defining how you will collaborate and which tasks you will carry out.



We all want to get to the code quickly, but the more time you spend on planning, the less time you waste on unpredictable problems.

## Project Construction

---

As part of this project, you will have to develop two independent parts: a backend and a mobile application or a web frontend.

To homogenize the project's construction, you will use Docker and Docker Compose. You must dockerize your backend and your frontend. If you choose to build a mobile client, your Docker image must handle the build of your APK.

## Build your project

---

You have to deliver a Docker Compose file at the root of your project. This file must include at least the following services:

- ✓ a **backend** service to launch the **backend server**;
- ✓ a **frontend** service to launch or build the **frontend**;
- ✓ a **database** service to launch the database;
- ✓ a **reverse proxy** service to:
  - allow access to the backend and frontend services;
  - expose a public port.



Check the documentation about **network**, **depends\_on**, and **multistage**.

Your project is intended to go into production: prepare the ground by creating a second Docker Compose file for this environment.



Building for production involves several aspects to take into consideration.

## **docker compose up**

---

Validation of the integrity of your images will be done when launching the **docker compose up** command. The following points must be respected:

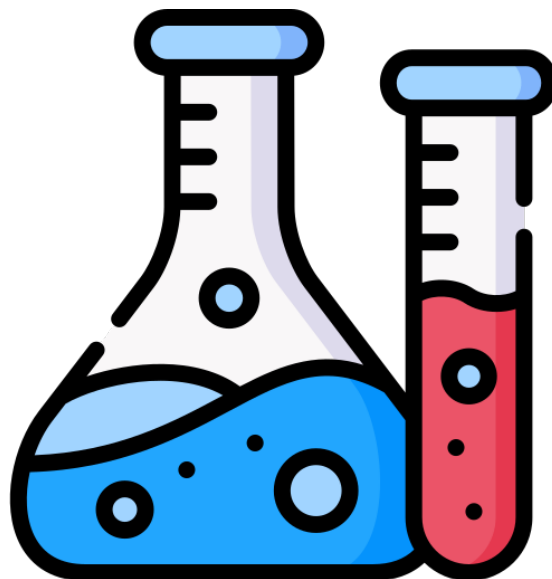
- ✓ if the client is a **mobile** application, its service must build and provide the mobile application;
- ✓ the **database** service must persist the data.

## **Testing**

---

To ensure the quality of a project, testing it is important.

To assert the behaviour of your backend application and avoid regressions, you're expected to write tests. Your backend, which is the core of your application and contains the business logic, must be tested. You're expected to test as many of your API routes as possible. Untested routes must be justified.



## **Pipeline**

---

You will work with GitHub as your source control management platform.



To facilitate collaboration, it's important to define your git flow and configure your repository accordingly: commit message norm, merge request configuration, ...



How to contribute to your project could be useful information to have in your README for other developers.

Working on your project, adding new features and ensuring quality require several repetitive manual tasks: building, launching tests, etc. Automation must be your new best friend, saving time and avoiding human errors.

Using **GitHub Actions**, you **MUST** create a pipeline with the following jobs:

- ✓ build your frontend and backend;
- ✓ launch some tests;
- ✓ security gates;
- ✓ create a test coverage report.

The logs of your CI must be managed carefully.



What is a mistake if everything is a mistake?

## Backend application

Your application will allow your employees to record their arrivals and departures, and their managers to handle the teams and see KPIs about company members.

Managing employees in teams and handling schedules require a lot of logic and need to be developed carefully, avoiding errors in the work-hours count that can lead to strikes.

Mainly, your application allows the employees of a company to:

- ✓ report their arrival and departure times;
- ✓ view a summary of their working hours.

As said earlier, the application will be composed of two main parts:

- ✓ the backend application;
- ✓ the frontend client.

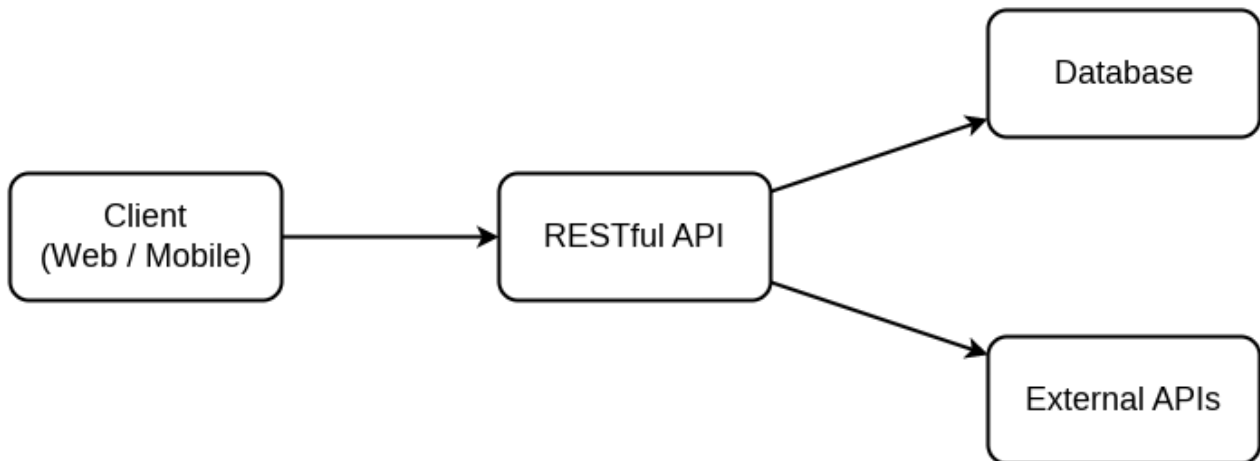
All the business logic and features of your application will be contained in your backend server.

The backend is the core of your application. The frontend will interact with it exclusively through an API.

## API

---

Your **A**pplication **P**rogramming **I**nterface must be RESTful.



The users' content (employees and managers) allows recording information such as:

- ✓ first name;
- ✓ last name;
- ✓ email;
- ✓ phone number.

The team's content allows recording some information about a team, such as:

- ✓ name;
- ✓ description;
- ✓ members;

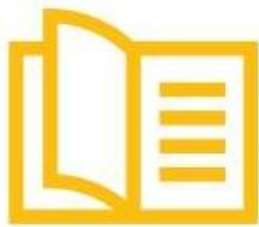
- ✓ manager.

You must implement, at least, the following features usable via endpoints:

- ✓ retrieve the users;
- ✓ add a user;
- ✓ update a user;
- ✓ delete a user;
- ✓ retrieve the teams;
- ✓ add a team;
- ✓ update a team;
- ✓ delete a team;
- ✓ set the arrival / departure of the authenticated user;
- ✓ get a summary of the arrivals and departures of an employee;
- ✓ get a global report based on the chosen KPIs.



**Create**



**Read**



**Update**



**Delete**

## Roles

---

The application is aimed at two kinds of users: **employees** and **managers**. These two roles will allow users to have access to specific features.

Below is a list of features that should be implemented:

- ✓ Common features:
  - edit their account information;
  - delete their account;
  - report their departure and arrival times;
  - view their dashboards.
- ✓ Manager features:
  - manage their team(s);
  - view the averages of the daily and weekly hours of the team over a given period;

- view the daily and weekly working hours of an employee over a period of time;
- view their employees' dashboards;
- register employees.



This list of features is non-exhaustive.

It is essential to adapt it according to your research and the various audits that you will carry out, so that the application corresponds to the needs of each category of users.

## KPI

---

You must choose at least 2 types of KPIs to help managers supervise the company.

## Authentication

---

As the application is centred on the digital life of users, it must offer a way to manage them. To do this, you must create a user management module.

The company asks non-identified users to be registered by the managers.

Each API endpoint must be **secured** when it requires authentication.

Using the application requires knowing the user in question. To do this, it is necessary to implement the following options:

- ✓ a method of user authentication via email / password;
- ✓ password management.

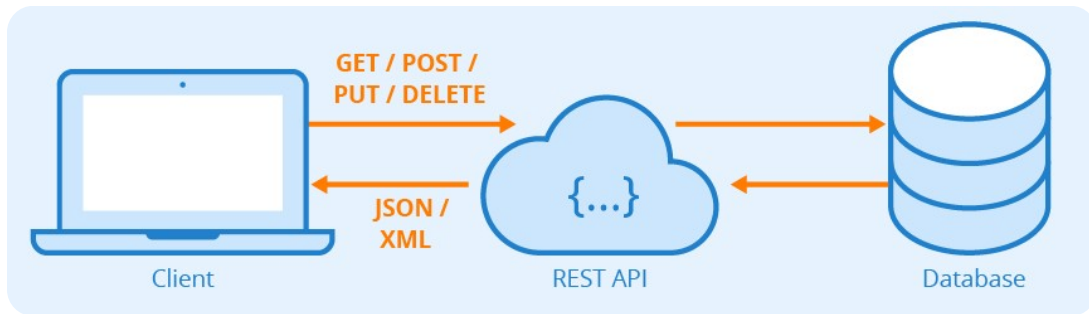


Ensure that basic security measures are in place to prevent credential theft.

The company uses Microsoft accounts for its employees. To make logging in easier, you must allow employees to log in with their Microsoft account.

## Frontend application

To develop the frontend, you have two options: build a mobile application or a web application. You must use the RESTful API you developed to implement the frontend.



## Interfaces

---

You must implement, at least, the following views:

- ✓ Authentication:
  - login;
- ✓ Users:
  - retrieve the users;
  - add a user;
  - update a user;
  - delete a user;
- ✓ Teams:
  - retrieve the teams;
  - add a team;
  - update a team;
  - delete a team;
- ✓ Time management:
  - set the arrival / departure of the authenticated user;
  - get a summary of the arrivals and departures of an employee;
  - get arrivals and departures;
- ✓ KPI:
  - get a global report based on the chosen KPIs.

## Role and security

---

There are two roles, named **employee** and **manager**. It is important to manage security on the pages according to the respective roles.

Of course, the pages must be protected to restrict access only to those who are authorized. Using the application requires knowing the user in question.

Below is a list of features that you should implement:

- ✓ Common features:
  - edit their account information;
  - delete their account;
  - report their departure and arrival times;
  - view their dashboards.
- ✓ Manager features:
  - manage their team(s);
  - view the averages of the daily and weekly hours of the team over a given period;
  - view the daily and weekly working hours of an employee over a period of time;
  - view their employees' dashboards;
  - register employees.



This list of features is non-exhaustive.  
You can add more if you find them relevant.



## UX & Accessibility Awareness

---

To ensure your application can be effectively used by all users, regardless of their context, device, or abilities, you must integrate good UX (User Experience) and Accessibility practices throughout your project.

Building an accessible and usable interface is not just a matter of compliance, it's a matter of quality,

inclusion, and efficiency.

Why it matters:

- ✓ A clear and consistent user experience increases adoption and reduces user errors.
- ✓ Accessibility ensures that people with disabilities (visual, motor, hearing, cognitive) can use your product.
- ✓ It improves SEO, usability, and overall quality perception.

Your project **must include accessibility and UX considerations** from the design phase to the CI pipeline.



a11y 🤔

## Documentation

Take the time to define a simple and scalable architecture, without code duplication.

You are expected to provide clear and simple documentation of your project.

The goal is to have a document that serves as a working support to easily understand the project in order to facilitate communication within teams and ease the integration of new developers.

Thus, there is no need to make class or sequence diagrams of the entire project, but rather to choose the important parts to understand what needs to be documented.



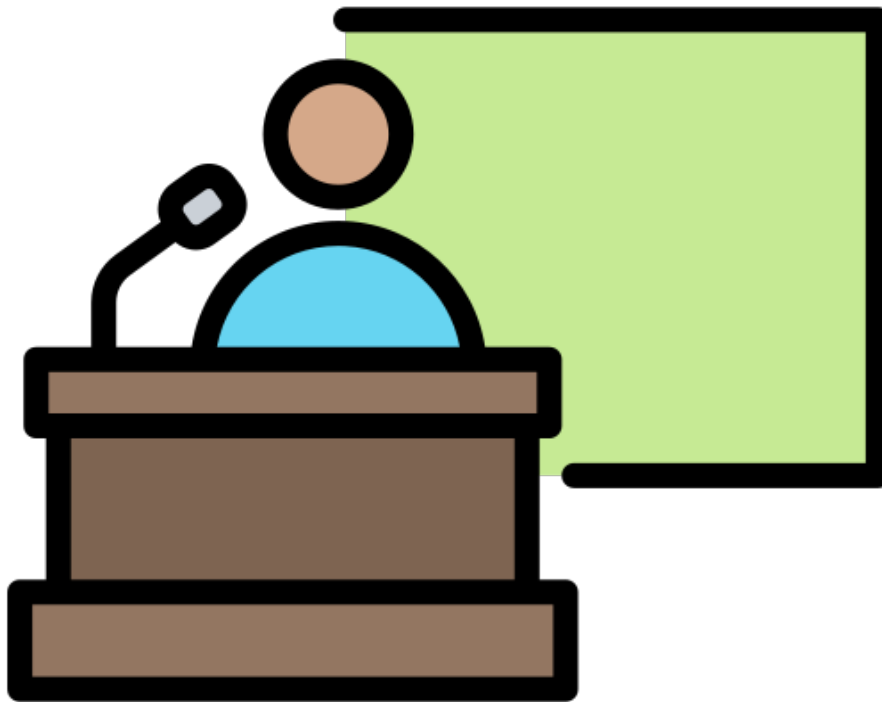
You must provide a README.md file describing your project and how to set it up for the launch.  
Too much information is bad information.



## Keynote

At the end of your project, you will have to:

- ✓ present the user needs you identified and how they shaped your features and KPIs;
- ✓ present the good practices that you have implemented to build your project;
- ✓ present your documentation;
- ✓ show your CI in action;
- ✓ defend your technology choices;
- ✓ show your code and architecture.



## Bonus

You can enhance your digital solution with several features:

- ✓ Add a physical device or module to clock employees in and out.
- ✓ Add an AI module to help managers analyze team performance.
- ✓ Create a 3D view of the office to help employees visualize their workspace.
- ✓ Create an advanced data dashboard with multiple KPIs, visualizations, and filters to help managers analyze activity.
- ✓ Design and deploy your solution using a cloud infrastructure.
- ✓ Add a multi-factor authentication system.

v 1.5.0

{EPITECH}